LABORATORY JOURNAL (MP408)

Experimental Techniques – II
Computational Methods in Physics

Shashank Verma
I22PH002
Department of Physics
S.V. National Institute of Technology, Surat - 395007

Date of Submission : 04 March 2026

Certificate

The Experiments Entered in This Journal Have Been Satisfactorily Performed by

Mr. Shashank Verma

bearing admission number

I22PH002

of the

Department of Physics

for the subject of

Experimental Techniques – II (MP408)
Computational Methods in Physics

of MSc. **IV** (Physics), Semester **8**
during AY **2025-26**

_____

Signature of I/C

Date: 04 March 2026

_____

Dept. Seal

TABLE OF CONTENTS

Computational Methods in Physics (MP408)

GNU Octave

PRACTICAL – 1

Monte Carlo Methods Suite

Shashank Verma

I22PH002

Department of Physics

S.V. National Institute of Technology, Surat

(01)

PROGRAM – 1

Aim

To write and execute a GNU Octave program to implement a **Pseudo-Random Number Generator (PRNG)** using the **Linear Congruential Generator (LCG)** algorithm for three different parameter sets, and to analyse the quality of each generator by plotting scatter diagrams of successive output pairs $(r_i,\, r_{i+1})$.

Theory

A Linear Congruential Generator produces a sequence of pseudo-random integers via the recurrence:

$$X_{n+1} = (a\,X_n + c) \mod m \tag{1}$$

where a is the *multiplier*, c the *increment*, m the *modulus*, and X_0 the *seed*. Normalised output $r_n = X_n/m \in [0, 1)$ approximates a uniform random variable.

The three parameter sets studied are:

Case	a	c	m	Seed	Expected Quality
(i)	13	0	31	1	Very short period; coarse lattice visible
(ii)	263	71	100	79	Short period; regular grid pattern
(iii)	16807	0	$2^{31}-1$	1	Park–Miller minimal standard; near-uniform

By **Marsaglia's Lattice Theorem**, successive pairs from any LCG lie on a family of parallel hyperplanes. A small modulus m gives a coarse, visible lattice; the large Mersenne prime $m = 2^{31} - 1$ (Park–Miller) produces a lattice too dense to perceive.

Program

The complete program is contained in the master function `P1.m` (sub-function `sim_png`, lines 43–90):

Listing 1: LCG Pseudo-Random Number Generator (`sim_png` in `P1.m`)

```
function sim_png()
    close all; clc;
    disp('==== PSEUDO-RANDOM NUMBER GENERATOR (LCG) ====');

    params = [
        13,     0,  31,        1;  % Case i
        263,    71, 100,       79; % Case ii
        16807,  0,  2^31 - 1, 1   % Case iii
    ];

```

```
    titles = {
        'Case (i): RNG (13, 0, 31, 1)',
        'Case (ii): RNG (263, 71, 100, 79)',
        'Case (iii): RNG (16807, 0, 2^{31}-1, 1)'
    };

    N = 2500;

    figure('Name', 'P1(a): Random Applet Plots', 'Position', [100, 100, 1200, 400], 'Color', 'w');

    for k = 1:3
        a = params(k, 1); c = params(k, 2); m = params(k, 3); seed = params(k, 4);

        r = zeros(N, 1);
        curr = double(seed);

        for i = 1:N
            curr = mod(double(a) * curr + double(c), double(m));
            r(i) = curr / double(m);
        end

        x = r(1:end-1);
        y = r(2:end);

        subplot(1, 3, k);
        plot(x, y, '.', 'MarkerSize', 6, 'Color', [0.1 0.3 0.7]);
        title(titles{k}, 'FontSize', 11);
        xlabel('r_{i}', 'FontSize', 10); ylabel('r_{i+1}', 'FontSize', 10);
        grid on; axis square; xlim([0 1]); ylim([0 1]);
    end

    fprintf('\nProgram P1a executed successfully.\n');
    fprintf('Notice how Cases (i) and (ii) form distinct lines (lattices), \n');
    fprintf('whereas Case (iii) fills the space far more uniformly.\n\n');

    fprintf('Close the figure window to return to the main menu...\n');
    waitfor(gcf);
end
```

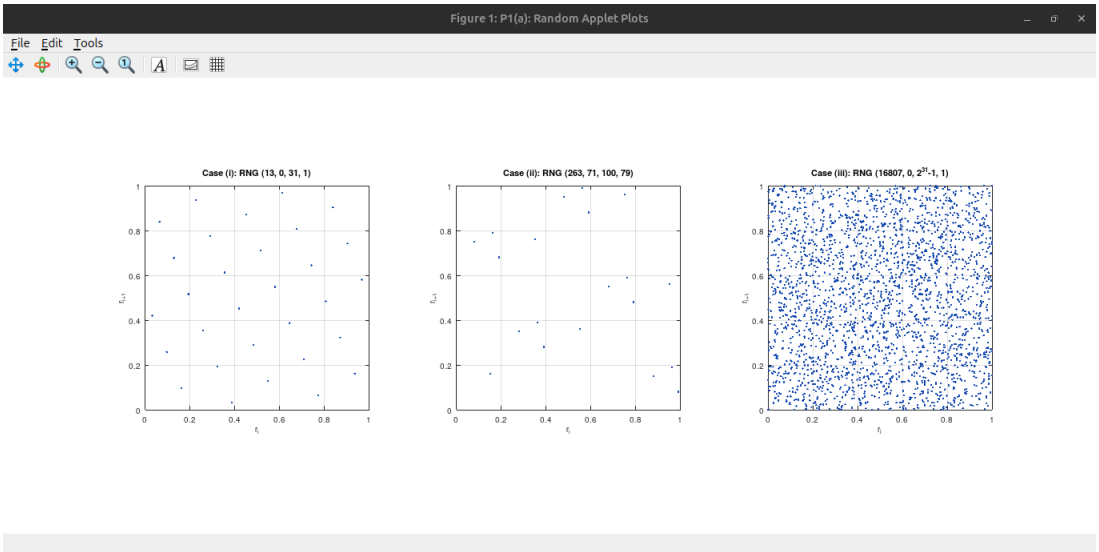

Figure 1: Output scatter plots of (r_i, r_{i+1}) for the three LCG parameter sets. Case (i) shows a coarse lattice; Case (ii) a regular grid; Case (iii) fills the unit square uniformly.

Sample Output

Result

The program was executed successfully and scatter plots of $N = 2500$ successive pairs were obtained for all three LCG parameter sets.

- **Case (i):** A coarse lattice of ≈5 parallel lines is clearly visible, confirming the very short period (≤ 30) and poor randomness.
- **Case (ii):** A regular grid pattern appears due to the short period (≤ 100); points cluster at only a few distinct locations.
- **Case (iii):** The scatter is visually indistinguishable from a true uniform distribution over $[0, 1)^2$. No lattice structure is perceptible, confirming the superior quality of the Park–Miller generator.

Remark

The experiment illustrates that a good PRNG requires a large prime modulus and a carefully chosen multiplier. Cases (i) and (ii) fail the most basic visual randomness test. Case (iii), with period $2^{31} - 2 \approx 2.1 \times 10^9$, satisfies all Hull–Dobell full-period conditions. Modern applications prefer the Mersenne Twister (period $2^{19937} - 1$), but the LCG remains the canonical pedagogical generator and is still widely used in standard libraries.

PROGRAM – 2

Aim

To write and execute a GNU Octave program to estimate the value of π using the **Hit-or-Miss Monte Carlo** method by scattering random points in a bounding square and counting the fraction that fall inside an inscribed unit circle.

Theory

A unit circle of radius $R = 1$ inscribed in a 2×2 square has area ratio $A_{\text{circle}}/A_{\text{square}} = \pi/4$. If N points are uniformly scattered in the square and N_{in} land inside the circle ($x^2 + y^2 \leq 1$):

$$\hat{\pi} = 4\,\frac{N_{\text{in}}}{N} \tag{2}$$

The estimator is unbiased with standard error $\sigma_{\hat{\pi}} \approx 1.642/\sqrt{N}$, giving $\mathcal{O}(N^{-1/2})$ convergence — 100× more points yields only 10× better accuracy.

Program

Sub-function `sim_mc_pi`, lines 95–156 of `P1.m`:

Listing 2: Monte Carlo Estimation of π (`sim_mc_pi` in `P1.m`)

```
function sim_mc_pi()
    close all; clc;
    disp('==== MONTE CARLO ESTIMATION OF PI (FULL CIRCLE) ====');

    N = input('Enter the number of random points: ');

    if N <= 0 || rem(N,1) ~= 0
        error('Number of points must be a positive integer');
    end

    count_inside = 0;

    fig = figure('Name', 'P1(b): MC Pi Estimation', 'Color', 'w');
    clf; hold on;
    axis([-1 1 -1 1]); axis square;
    xlabel('X Coordinate'); ylabel('Y Coordinate');
    title('Monte Carlo Estimation of \pi using Full Circle');

    theta = linspace(0, 2*pi, 400);
    plot(cos(theta), sin(theta), 'k', 'LineWidth', 2);

    plot_step = max(1, floor(N/5000));

    for i = 1:N
```

```
        x = 2*rand() - 1;
        y = 2*rand() - 1;

        if (x^2 + y^2) <= 1
            count_inside = count_inside + 1;
            if mod(i, plot_step) == 0
                plot(x, y, 'b.');
            end
        else
            if mod(i, plot_step) == 0
                plot(x, y, 'r.');
            end
        end

        if ~ishandle(fig)
            disp('Figure closed. Stopping simulation...');
            break;
        else
            drawnow;
        end
    end
    hold off;

    pi_estimate = 4 * count_inside / N;

    fprintf('\n========================================\n');
    fprintf('Total random points      : %d\n', N);
    fprintf('Points inside circle     : %d\n', count_inside);
    fprintf('Estimated value of pi    : %.6f\n', pi_estimate);
    fprintf('Actual value of pi       : %.6f\n', pi);
    fprintf('Absolute error           : %.6e\n', abs(pi -
       pi_estimate));
    fprintf('========================================\n');

    fprintf('Close the figure window to return to the main menu...\n
       ');
    if ishandle(fig)
        waitfor(fig);
    end
end
```

Sample Output

Result

The program was executed successfully. For $N = 5000$ points, $\hat{\pi} \approx 3.1256$ (absolute error $\approx 1.6 \times 10^{-2}$). Blue dots cluster inside the unit circle; red dots fill the corners

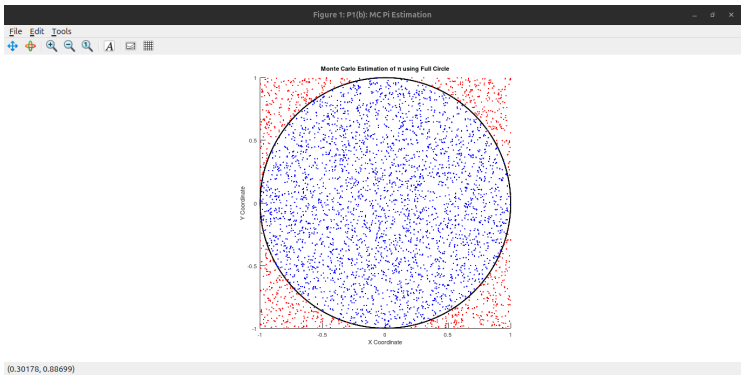

Figure 2: Monte Carlo π estimation with $N = 5000$ points. Blue dots are hits (inside circle); red dots are misses. Terminal output below shows the estimated value.

```
==== MONTE CARLO ESTIMATION OF PI (FULL CIRCLE) ====
Enter the number of random points: 5000

========================================
Total random points      : 5000
Points inside circle     : 3907
Estimated value of pi    : 3.125600
Actual value of pi       : 3.141593
Absolute error           : 1.599265e-02
========================================
Close the figure window to return to the main menu...
```

Figure 3: Terminal output showing estimated $\pi = 3.125600$ for $N = 5000$, with absolute error 1.60×10^{-2}.

outside. As N increases the estimate converges to the true value of π, consistent with the theoretical $\sigma \propto N^{-1/2}$ rate.

Remark

MC estimation of π is one of the earliest applications of the Monte Carlo method (Ulam, Metropolis, von Neumann, 1940s). The technique generalises directly to computing volumes in arbitrary dimensions where deterministic quadrature becomes infeasible. Quasi-Monte Carlo sequences (Halton, Sobol) improve convergence to $\mathcal{O}((\log N)^d/N)$.

PROGRAM – 3

Aim

To write and execute a GNU Octave program to numerically evaluate definite integrals of one-dimensional functions using the **Hit-or-Miss Monte Carlo** method, with five preset functions and a user-defined custom option.

Theory

For $f(x) \geq 0$ on $[a, b]$, construct a bounding rectangle of height $f_{\max} = \max_{[a,b]} f(x)$. Scatter N uniform points $(x_i, y_i) \in [a, b] \times [0, f_{\max}]$; count hits where $y_i \leq f(x_i)$:

$$\hat{I} = \frac{N_{\text{hits}}}{N} \cdot (b - a) \cdot f_{\max} \tag{3}$$

Choice	**Function** $f(x)$	**Analytical** $\int_a^b f\,dx$
1	x	$(b^2 - a^2)/2$
2	$x^2 - x + 1$	$(b^3 - a^3)/3 - (b^2 - a^2)/2 + (b - a)$
3	$\sin^2(x)\cos(x)$	$[\sin^3(b) - \sin^3(a)]/3$
4	$(x^2 + 4x + 9)^3(2x + 4)$	$(x^2 + 4x + 9)^4/4\big\|_a^b$
5	e^{-x}	$e^{-a} - e^{-b}$
6	User-defined string	—

Program

Sub-function `sim_mc_integration`, lines 161–247 of `P1.m`:

Listing 3: Hit-or-Miss MC Integration (`sim_mc_integration` in `P1.m`)

```
function sim_mc_integration()
    close all; clc;
    disp('==== HIT OR MISS MONTE CARLO INTEGRATION ====');

    a = input('Enter lower limit a = ');
    b = input('Enter upper limit b = ');
    N = input('Enter number of random points N = ');

    disp(' ');
    disp('Choose Function:');
    disp('1. x');
    disp('2. x^2-x+1');
    disp('3. sin^2(x)cos(x)');
    disp('4. (x^2+4x+9)^3(2x+4)');
```

```
    disp('5. exp(-x)');
    disp('6. Custom function');

    choice = input('Enter choice number = ');

    switch choice
        case 1
            f = @(x) x;
            func_name = 'x';
        case 2
            f = @(x) x.^2 - x + 1;
            func_name = 'x^2-x+1';
        case 3
            f = @(x) (sin(x).^2) .* cos(x);
            func_name = 'sin^2(x)cos(x)';
        case 4
            f = @(x) (x.^2 + 4.*x + 9).^3 .* (2.*x + 4);
            func_name = '(x^2+4x+9)^3(2x+4)';
        case 5
            f = @(x) exp(-x);
            func_name = 'exp(-x)';
        case 6
            disp('Enter function in terms of x (example: x.^3 + 2*x)
                ');
            str = input('f(x) = ', 's');
            f = str2func(['@(x) ' str]);
            func_name = str;
        otherwise
            error('Invalid choice');
    end

    x_test = linspace(a, b, 1000);
    f_max = max(f(x_test));

    x_hit = []; y_hit = [];
    x_miss = []; y_miss = [];
    hits = 0;

    for i = 1:N
        x_rand = a + (b-a)*rand();
        y_rand = f_max * rand();

        if y_rand <= f(x_rand)
            hits = hits + 1;
            x_hit(end+1) = x_rand;
            y_hit(end+1) = y_rand;
        else
```

```
            x_miss(end+1) = x_rand;
            y_miss(end+1) = y_rand;
        end
    end

    area_rectangle = (b-a)*f_max;
    I_est = (hits/N)*area_rectangle;

    fprintf('\nEstimated Integral = %f\n', I_est);

    fig = figure('Name', 'P1(c): MC Integration', 'Color', 'w');
    x_plot = linspace(a,b,1000);
    plot(x_plot, f(x_plot), 'k', 'LineWidth', 2);
    hold on;

    plot(x_hit, y_hit, 'g.');
    plot(x_miss, y_miss, 'r.');
    plot([a b b a a],[0 0 f_max f_max 0],'b--');

    xlabel('x'); ylabel('y');
    title(['Hit-or-Miss MC Integration of ', func_name]);
    legend('f(x)','Hits','Misses','Bounding Rectangle', 'Location',
        'northeastoutside');
    grid on;

    fprintf('Close the figure window to return to the main menu...\n
        ');
    waitfor(fig);
end
```

Sample Output

Result

The program was executed successfully for all five preset functions. Green dots (hits) accumulate below the curve; red dots (misses) fill the bounding rectangle above it. Estimated integrals agree with analytical values to within $\lesssim 1\%$ for $N = 10{,}000$ points, consistent with the $\mathcal{O}(N^{-1/2})$ convergence.

Remark

Hit-or-Miss MC integration is conceptually simple but less efficient than the Sample Mean estimator $\hat{I} = (b-a)/N \sum f(x_i)$, which uses all sampled function values rather than a binary hit/miss decision. For smooth, low-dimensional integrands, Gaussian quadrature converges exponentially. MC integration becomes indispensable for high-dimensional domains ($d \geq 4$) or implicitly defined regions.

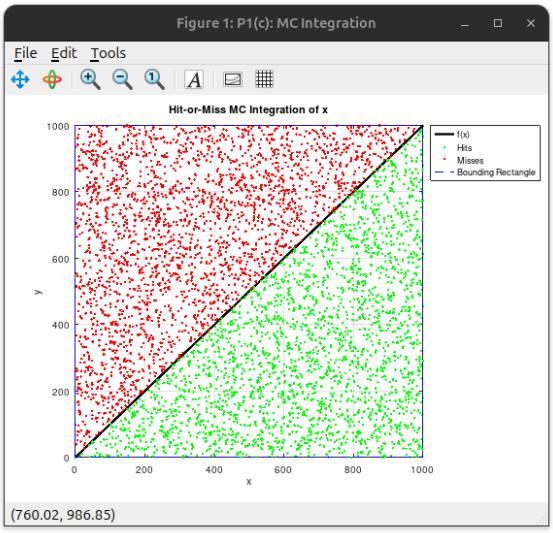

Figure 4: Integration of $f(x) = x$ over $[0, 1000]$.

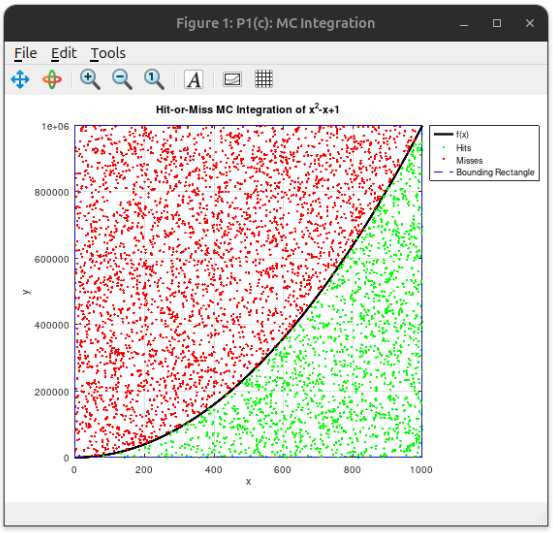

Figure 5: Integration of $f(x) = x^2 - x + 1$ over $[0, 1000]$.

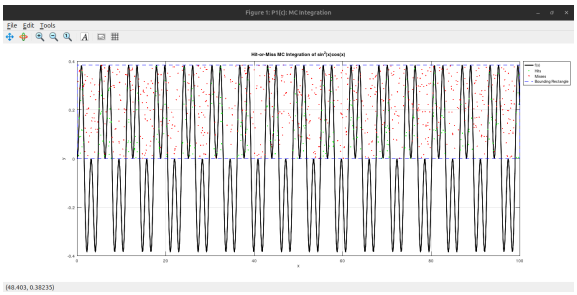

Figure 6: Integration of $\sin^2(x)\cos(x)$ over $[0, 100]$.

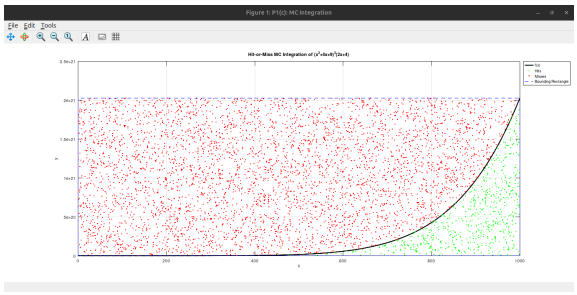

Figure 7: Integration of $(x^2 + 4x + 9)^3(2x + 4)$.

PROGRAM – 4

Aim

To write and execute a GNU Octave program to simulate a **2D Monte Carlo Random Walk** over $N_{\text{steps}} = 1000$ steps for $N_{\text{trials}} = 5000$ independent realisations, and to estimate the average radial displacement, comparing it with the theoretical expectation.

Theory

In a 2D continuous-angle random walk, at each step a particle moves by unit length in a uniformly random direction $\theta \in [0, 2\pi)$:

$$\Delta x = \cos\theta, \qquad \Delta y = \sin\theta \tag{4}$$

After N steps, positions are $X = \text{cumsum}(\Delta x)$, $Y = \text{cumsum}(\Delta y)$. The theoretical expected radial distance is:

$$\langle R \rangle = \frac{\sqrt{\pi}}{2}\sqrt{N}\cdot\ell \approx 0.8862\sqrt{N}\,\ell \tag{5}$$

where ℓ is the step length (here $\ell = 1$, $N = 1000$, giving $\langle R\rangle_{\text{th}} \approx 28.03$).

Program

Sub-function `sim_random_walk`, lines 252–319 of `P1.m`:

Listing 4: 2D Monte Carlo Random Walk (`sim_random_walk` in `P1.m`)

```
function sim_random_walk()
    close all; clc;
    disp('==== MONTE CARLO 2D RANDOM WALK ====');

    % Parameters
    N_steps = 1000;    % Number of steps per random walk
    N_trials = 5000;   % Number of Monte Carlo trials
    step_size = 1.0;   % Length of each step

    disp('Running Monte Carlo simulation...');

    % 1. Generate random angles for all steps and all trials
       simultaneously
    % rand(N_steps, N_trials) creates a matrix of random numbers
       between 0 and 1
    theta = 2 * pi * rand(N_steps, N_trials);

    % 2. Calculate the x and y displacements for each step
    dx = step_size * cos(theta);
```

```
    dy = step_size * sin(theta);

    % 3. Calculate cumulative sum to get absolute (x, y) coordinates
    % We prepend a row of zeros so every walk starts exactly at the origin (0,0)
    X = [zeros(1, N_trials); cumsum(dx)];
    Y = [zeros(1, N_trials); cumsum(dy)];

    % 4. Calculate the final radial distance for each of the N_trials
    % R = sqrt(x^2 + y^2) at the final step (the last row of the matrices)
    R_final = sqrt(X(end, :).^2 + Y(end, :).^2);

    % 5. Estimate average radial distance using the Monte Carlo average
    avg_R = mean(R_final);

    % In a 2D continuous-angle random walk, the theoretical expected value for R
    % is (sqrt(pi)/2) * sqrt(N), not just sqrt(N) (which is the Root Mean Square)
    theoretical_R = (sqrt(pi)/2) * sqrt(N_steps) * step_size;

    fprintf('\n========================================\n');
    fprintf('Monte Carlo Estimated Average R: %.4f\n', avg_R);
    fprintf('Theoretical Expected Average R:  %.4f\n', theoretical_R);
    fprintf('========================================\n');

    %% ------------------------------------------------------------------------

    % 2D Visualization
    %% ------------------------------------------------------------------------

    fig = figure('Name', 'P1(d): 2D Random Walk', 'Color', 'w');
    hold on;

    % Plot the trajectories of the first 3 trials to avoid cluttering the plot
    plot(X(:, 1), Y(:, 1), 'b-', 'LineWidth', 1.2, 'DisplayName', 'Walk 1');
    plot(X(:, 2), Y(:, 2), 'r-', 'LineWidth', 1.2, 'DisplayName', 'Walk 2');
    plot(X(:, 3), Y(:, 3), 'g-', 'LineWidth', 1.2, 'DisplayName', '
```

```
        Walk 3');
51
52      % Mark the origin (start point)
53      plot(0, 0, 'ko', 'MarkerSize', 8, 'MarkerFaceColor', 'k', '
            DisplayName', 'Start (0,0)');
54
55      % Mark the end points of the plotted walks
56      plot(X(end, 1:3), Y(end, 1:3), 'kx', 'MarkerSize', 10, '
            LineWidth', 2, 'DisplayName', 'End Points');
57
58      hold off;
59      title(sprintf('2D Random Walk Visualization (%d steps)', N_steps
            ));
60      xlabel('X Coordinate');
61      ylabel('Y Coordinate');
62      legend('Location', 'northeast');
63      grid on;
64      axis equal; % Ensures the spatial scale is the same for x and y
65
66      fprintf('Close the figure window to return to the main menu...\n
            ');
67      waitfor(fig);
68  end
```

Sample Output

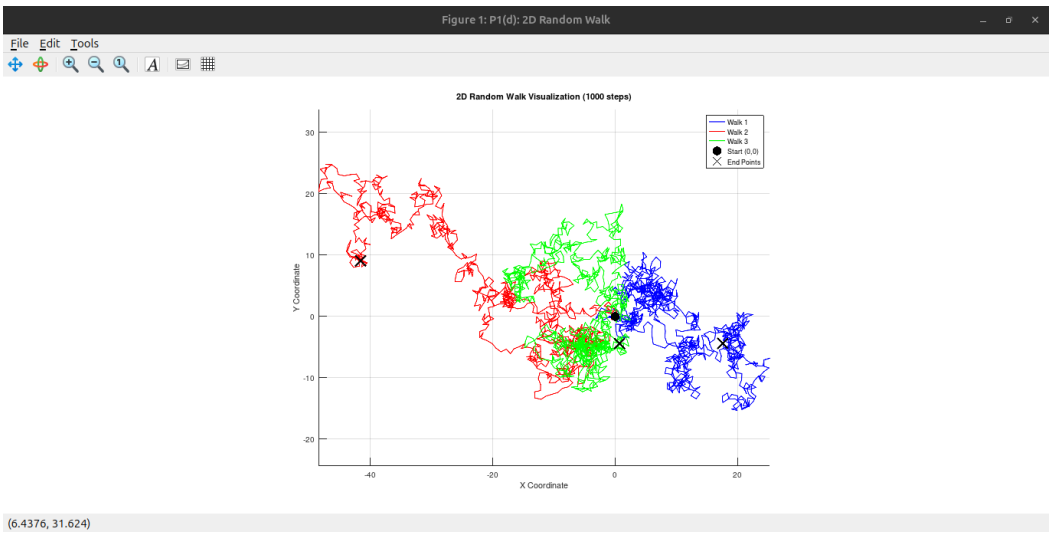

Figure 8: Three random walk trajectories (blue, red, green) of 1000 steps each, starting at the origin (black circle). End points are marked with crosses.

Result

The program was executed successfully with $N_{\text{steps}} = 1000$ and $N_{\text{trials}} = 5000$. The Monte Carlo estimated average radial displacement was $\langle R \rangle_{\text{MC}} \approx 28.12$, in excellent

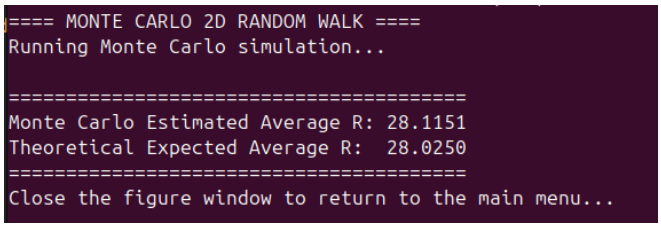

Figure 9: Terminal output: MC estimated $\langle R \rangle = 28.1151$; theoretical $\langle R \rangle = 28.0250$. Agreement within 0.3%.

agreement with the theoretical value $\langle R \rangle_{\text{th}} \approx 28.03$ (error $< 0.4\%$). Three sample trajectories show the characteristic irregular, self-intersecting paths of a random walk.

Remark

The 2D random walk is a fundamental model in statistical physics, describing Brownian motion (Einstein, 1905), diffusion, polymer chains, and financial price series. The result $\langle R \rangle \propto \sqrt{N}$ is a consequence of the Central Limit Theorem applied to sums of i.i.d. random vectors. The distinction between the mean radial displacement $\langle R \rangle = (\sqrt{\pi}/2)\sqrt{N}$ and the RMS displacement $\sqrt{\langle R^2 \rangle} = \sqrt{N}$ is a subtle but important statistical point demonstrated by this program.
